

CSCE 611 OPERATING SYSTEMS · MODULE 09 · LECTURE NOTES

Crash Consistency

Surviving a power failure in the middle of an update.

MODULE OUTCOMES

- Explain why a crash mid-update corrupts a file system.
- Explain fsck and why it is inadequate.
- Explain write-ahead logging and why ordering is everything.
- Compare journaling modes and their guarantees.
- Explain copy-on-write file systems as an alternative.

1 Why a crash corrupts

A single logical operation — appending a block to a file — requires several physical writes that must all happen or none.

Appending one block requires:

1. mark the block allocated in the free bitmap
2. record the block number in the inode
3. write the data into the block itself

Crash after	State	Severity
step 1 only	A block is marked used and referenced by nothing.	A leak . Space is wasted until fsck reclaims it. Annoying, not dangerous.
step 2 only	The inode references a block that the bitmap says is free. That block will eventually be allocated to another file.	Corruption . Two files sharing one block; writes through one appear in the other.
steps 1 and 2	The inode references an allocated block containing whatever was there previously.	Security failure . The file now contains the previous contents of that block — possibly another user's deleted data.

THE ONLY PRIMITIVE YOU HAVE

A single sector write is atomic: it completes entirely or not at all. The device guarantees that much and very little else.

Writes are reordered. Issuing A then B does not mean A reaches durable storage first; the device's queue may reorder freely for performance.

Write caches may lie. Some devices acknowledge a write before it is durable.

So the engineering problem is to build a multi-block atomic update out of one atomic sector write plus explicit ordering barriers. Everything in this module is a way of doing that.

2 fsck and its limits

The historical approach was to repair afterwards. On boot, fsck walks the entire file system, rebuilds the free bitmap from what the inodes actually reference, corrects link counts, and attaches unreferenced inodes to lost+found.

- **It is slow.** The time is proportional to the size of the file system, not to the amount of work that was in progress when the crash occurred. A large volume takes hours, during which the system is unavailable — which became untenable as disks grew.
- **It restores consistency, not correctness.** This is the deeper problem.

CONSISTENCY IS NOT CORRECTNESS

fsck can detect that an inode references a block the bitmap marks free. It cannot determine which of the two is right, because that information was never written down.

It will choose one — typically trusting the inode and marking the block used — and produce a file system that is internally consistent. Whether it contains your data is a separate question it cannot answer.

A file system that passes fsck is one where the metadata agrees with itself. That is a much weaker property than 'your data is intact', and conflating the two has caused a great deal of misplaced confidence.

3 Write-ahead logging

The insight is to record *what you are about to do* somewhere safe before doing it, so that after a crash you can either complete or abandon the operation, but never be caught halfway.

1. Write all new block contents to the LOG.
2. BARRIER -- wait until those writes are durable.
3. Write a single-sector COMMIT record.
4. BARRIER.
5. Write the blocks to their real locations (checkpoint).
6. Mark the log entry free.

RECOVERY: scan the log.

committed transactions	-> replay (idempotent)
uncommitted	-> discard

ONE ATOMIC SECTOR MAKES A MULTI-BLOCK UPDATE ATOMIC

The commit record is a single sector, so writing it is atomic: it is either there or it is not.

Before the commit record lands, recovery discards the transaction and the file system is exactly as it was. After it lands, recovery replays the transaction and the file system is exactly as if the operation had completed. There is no third state.

Replay must be **idempotent** — a crash during recovery must be survivable — which it is, because replay simply writes known block contents to known locations.

THE BARRIERS ARE THE MECHANISM, NOT AN OPTIMISATION DETAIL

Suppose the commit record reached durable storage before the logged data it describes. A crash in between leaves a committed transaction whose data is absent, and recovery faithfully replays garbage into the file system — worse than having done nothing.

Because devices reorder, issuing the writes in order is insufficient. The file system must **wait** for the data to be durable before even issuing the commit. That is step 2, and it is where journaling's cost lies.

Through the 2000s, several file systems and drive firmwares omitted or weakened these barriers for benchmark performance, and lost user data as a direct result. The ordering requirement is not negotiable, and 'faster' implementations that drop it are simply incorrect.

3.1 Modes

Mode	What is journaled	Guarantee	Cost
Data	Metadata and file data.	Full: after recovery, metadata is consistent and data is either the old version or the new one.	All data written twice. Roughly halves write throughput.
Ordered	Metadata only, but data blocks are forced to disk <i>before</i> the metadata that references them is committed.	Metadata consistent, and a file never references a block containing stale contents.	Moderate. The default in ext4 for good reason.

Mode	What is journaled	Guarantee	Cost
Writeback	Metadata only, with no ordering constraint on data.	Metadata consistent. A file may reference blocks whose contents were never written — exposing whatever was there before.	Fastest, and a security hazard.

Writeback mode's failure is concrete: after a crash, a newly extended file can contain the previous contents of its blocks — potentially another user's deleted data. That is why ordered mode is the default despite being slower.

4 Copy-on-write file systems

A different answer: never overwrite a live block at all.

- 1 To modify a block, write a **new copy** at a free location.
- 2 The parent block that pointed at it must now be updated — so write a new copy of the parent too.
- 3 This propagates up the tree to the root.
- 4 Finally, one atomic single-sector write switches the **root pointer** to the new tree.

THERE IS NO INTERMEDIATE STATE

Before the root pointer is updated, the entire old tree is intact and valid. After, the entire new tree is intact and valid. A crash at any moment leaves one or the other.

Recovery is therefore trivial: there is nothing to do. The root pointer either references the old tree or the new one, and both are consistent.

Snapshots come free. Retain an old root pointer and you have a complete, consistent, read-only view of the file system as it was at that instant, sharing all unmodified blocks with the present. This is why ZFS, btrfs, and APFS make snapshots nearly costless while ext4 has to construct them with separate machinery.

	Journaling	Copy-on-write
Data written	Twice in data mode; once in ordered mode.	Once, to a new location.
Layout over time	Preserved — blocks stay where they were.	Fragments. Repeated small updates scatter a file, and performance degrades as the pool fills.
Snapshots	Require separate machinery (LVM, device mapper).	Intrinsic and nearly free.
Recovery	Scan and replay the log.	Nothing to do; the root pointer is always valid.
Checksums	Usually metadata only.	Typically end-to-end on data as well, detecting silent corruption.
Examples	ext4, XFS, NTFS, JFS.	ZFS, btrfs, APFS, NetApp WAFL.

5 The idea generalises

LOG-THEN-COMMIT IS ONE OF THE FEW GENUINELY GENERAL IDEAS

Databases use write-ahead logging for transaction durability, with the same protocol and the same recovery procedure. The ARIES algorithm is this module, elaborated.

Distributed consensus — Raft and Paxos — is a replicated log with an agreed commit point. 'Committed' means the same thing: the point after which the operation definitely happened.

Git is a copy-on-write store. Objects are immutable, and a commit is an atomic update of a reference to point at a new tree — structurally identical to §4.

Copy-on-write fork (Module 02) is the same idea applied to memory pages.

The pattern is: *write the intent somewhere durable, then make it visible with a single atomic step.* Recognising it is worth more than any one of the instances.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapter 42 (Crash Consistency: FSCCK and Journaling)	The clearest written treatment, including the three-write example and the journaling modes.
xv6 book — Chapter 8, logging section	A complete, working journaling layer in a few hundred lines. Read it before implementing your own.
Rosenblum & Ousterhout — The Design and Implementation of a Log-Structured File System (free)	The log-structured approach, which treats the whole disk as a log. Influential well beyond file systems.
Bonwick & Moore — ZFS: The Last Word in File Systems (free slides)	Copy-on-write, end-to-end checksums, and snapshots from the designers.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read xv6's log.c. Identify the commit point and the two barriers, and explain what each protects against.
- 2 Build a crash-injection harness: run a file system operation in QEMU and kill the emulator at a randomly chosen point. Verify consistency afterwards.
- 3 Disable xv6's logging and run your crash injector a hundred times. Classify the resulting corruptions against the three cases in §1.

- 4 Restore logging and repeat. Report how many runs required recovery and how many left the file system inconsistent.
- 5 Deliberately reorder the commit record before the data writes and demonstrate that recovery now replays garbage.
- 6 Measure the cost of journaling: time a workload with logging enabled and disabled, and report the overhead.
- 7 Implement a trivial copy-on-write store: a tree of immutable blocks with an atomic root pointer. Demonstrate that a crash mid-update leaves either the old or the new tree, and implement snapshots by keeping old roots.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the three physical writes needed to append a block and the consequence of crashing after each.
- 2 What is the only atomicity guarantee a disk provides, and what else does it *not* guarantee?
- 3 Why is fsck inadequate, in two distinct respects?
- 4 State the write-ahead logging protocol, including both barriers.
- 5 Why must the commit record be a single sector?
- 6 Give the three journaling modes with the guarantee and cost of each, and explain writeback's security problem.
- 7 How does copy-on-write avoid needing recovery at all, and what does it cost?